

DSA 150-

Rotate Array

```
void rotate(vector<int>& nums, int k) {
    int n=nums.size();
    k%=n;
    reverse(nums.begin(),nums.end());
    reverse(nums.begin(),nums.begin()+k);
    reverse(nums.begin()+k,nums.end());
    // return nums;
}
```

Product of Array Except Self

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n=nums.size();
    vector<int>pre(n,1);
    vector<int>suff(n,1);
    vector<int>ans(n,1);
    for(int i=1;i<n;i++){
        pre[i]=pre[i-1]*nums[i-1];
    }
    for(int i=n-2;i>=0;i--){
        suff[i]=suff[i+1]*nums[i+1];
    }
    for(int i=0;i<n;i++){
        ans[i]=pre[i]*suff[i];
    }
    return ans;
}
```

Reverse Words in a String

```
string reverseWords(string s) {
    stringstream ss(s);
    string word;
    vector<string>words;
    while(ss>>word){
        words.push_back(word);
    }
    string ans;
    reverse(words.begin(),words.end());
    for(auto x:words){
        ans+=x+" ";
    }
    ans.pop_back();
    return ans;
}

while(ss >> word){
    words.push_back(word);
}
```

Iterations:

Iteration	Extracted word	words vector
1	"the"	["the"]
2	"sky"	["the","sky"]
3	"is"	["the","sky","is"]
4	"blue"	["the","sky","is","blue"]

Length of Last Word

```
int lengthOfLastWord(string s) {
    stringstream ss(s);
    string word;
    vector<string>v1;
    //int
    while(ss>>word){
        v1.push_back(word);
    }
    return v1.back().size();
}
```

Find the Index of the First Occurrence in a String

```
int strStr(string haystack, string needle) {
    int n=haystack.size();
    int m=needle.size();
    for(int i=0;i<=(n-m);i++){
        if(haystack.substr(i,m)==needle){
            return i;
        }
    }
    return -1;
}
```

Is Subsequence

```
bool isSubsequence(string s, string t) {
    int n=s.length();
    int m=t.length();
    int i=0;
    int j=0;
    while(i<n&&j<m){
        if(s[i]==t[j]){
            i++;
        }
        j++;
    }
    return (i==s.length());
}
```

Two Sum II - Input Array Is Sorted

```

/* vector<int>v1;
   int n=nums.size();
   for(int i=0;i<n;i++){
       for(int j=i+1;j<n;j++){
           if(nums[i]+nums[j]==target){
               return {i+1,j+1};
           }
       }
   }
   return {};
*/
int n=nums.size();
int i=0;
int j=n-1;
vector<int>v1;
while(i<j){
    if(nums[i]+nums[j]==target){
        v1.push_back(i+1);
        v1.push_back(j+1);
        break;
    }
    else if(nums[i]+nums[j]>target){
        j--;
    }
    else{
        i++;
    }
}
return v1;
}
};

```

BIT MANIPULATION-

Decimal to binary conversion-

```
int binary[32];
```

```
int i = 0;
```

```

while (n > 0) {
    binary[i] = n % 2;
    n = n / 2;
    i++;
}

```

```
cout << "Binary: ";
for (int j = i - 1; j >= 0; j--) {
    cout << binary[j];
}
```

```
string binary = "";
while (n > 0) {
    binary = char((n % 2) + '0') + binary;
    n = n / 2;
}
cout << "Binary: " << binary;
```

example:

n=13;

13/2=6 rem 1;

6/2=3 rem 0;

3/2=1 rem 1;

1/2=0 rem 1

Ans: 1101

BINARY TO DECIMAL CONVERSION-

```
int sum= 0, base= 1;
```

```
while (n > 0) {
    int lastDigit = n % 10;
    sum+= lastDigit * base;
    base *= 2;
    n /= 10;
}
```

```
cout << "Decimal: " << decimal;
```

```
decimal += lastDigit * base;
base *= 2;
```

1101:

1*2⁰=1

0*2¹=0

1*2²=4

1*2³=8

Sum=1+0+4+8=13

Toggle = switch between two states

Toggle Character (uppercase $\leftrightarrow$ lowercase)

```
int main() {  
    char ch = 'A';  
    ch = ch ^ 32;  
    cout << ch; // Output: a  
}
```

Take:

```
N = 10    // binary: 1010  
i = 1     // we toggle 1st bit (from right, 0-based)
```

◆ Step 1: Create mask

```
1 << i = 1 << 1 = 2    // binary: 0010
```

◆ Step 2: XOR operation

N	=	1010	(10)
mask	=	0010	(2)

result	=	1000	(8)

Check i-th bit

```
if (N & (1 << i));
```

Set i-th bit (make it 1)

```
N = N | (1 << i);
```

Clear i-th bit (make it 0)

```
N = N & ~(1 << i);
```

Shortcut Memory Trick

- $1 \ll i \rightarrow$ creates **mask**
- Then:
 - $\& \rightarrow$ check
 - $| \rightarrow$ set
 - $\wedge \rightarrow$ toggle
 - $\sim \rightarrow$ clear

Check if number is Power of 2

```
(n & (n - 1)) == 0
```

Check Odd or Even

```
if((n&1)==0) -->even
```

count the no. of set bits-

```
while(n){  
  
n&=(n-1);  
  
count++;  
}
```

__builtin_popcount()-

__builtin_popcountll()- builtin function for count set built

This sets the **rightmost 0 bit to 1**

```
n = n | (n + 1);
```

Single Number

```
int xorr=0;  
for(int i=0;i<nums.size();i++){  
    xorr^=nums[i];  
}  
return xorr;
```

Single Number III

```
int n=nums.size();  
int xorr=0;  
for(int i=0;i<n;i++){ //this will remove all the duplicates  
    xorr^=nums[i];  
}  
//unsigned int mask = (unsigned int)xorr & (-(unsigned int)xorr);  
int mask =xorr & (-(unsigned int)xorr);  
  
int num1=0;  
int num2=0;
```

```

for(auto x:nums){
    if((x&mask)==0){
        num1^=x;
    }
    else{
        num2^=x;
    }
}
return {num1,num2};
}

```

Missing Number

```

int missingNumber(vector<int>& nums) {
    int n=nums.size();
    int ans=0;
    for(int i=0;i<n;i++){
        ans^=i^nums[i];
    }
    ans^=n;
    return ans;
}

```

Nums=[3, 0, 1]

Idx=[0,1,2]

Ans=0^3^1^0^2^1

Remaining=3^2

Now do with n;

Remaining :3^2^3=2;

Subsets

```

class Solution {
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        int n=nums.size();
        int subset=1<n;
        vector<vector<int>> result;
        for(int i=0;i<subset;i++){
            vector<int> ans;
            for(int j=0;j<n;j++){
                if(i&(1<j)){
                    ans.push_back(nums[j]);
                }
            }
        }
    }
}

```

```

    }
    result.push_back(ans);
}
return result;
}
};

```

Swap Without Temp

```

a ^= b;
b ^= a;
a ^= b;

```



XOR of a Number Range

```

int xorrr(int n){
    if(n%4==1) return 1;
    else if(n%4==3) return 0;
    else if(n%4==0) return n;
    else return n+1;
}

int findXOR(int l, int r) {
    return xorrr(l-1)^xorrr(r);
}

```

Minimum Moves to Equal Array Elements II

When step size is 1 -

```

sort(nums.begin(), nums.end());
int median=nums[nums.size()/2];
int moves=0;
for(auto x:nums){
    moves+=abs(x-median);
}
return moves;

```

Minimum Operations to Make a Uni-Value Grid

When step size is n - $\text{moves} += \text{abs}(x - \text{median}) / n;$

Use:


```

int rem = arr[0] % x;
for (auto num : arr) {
    if (num % x != rem) return -1;
}

```

☞ When operation is like:

- num = num + x
- num = num - x

• Kadanes questions/prefix-suffix

• Maximum Subarray

```

class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int n=nums.size();
        int curr=0;
        int mxsum=INT_MIN;
        for(int i=0;i<n;i++){
            curr=max(nums[i],curr+nums[i]);
            mxsum=max(mxsum,curr);
        }
        return mxsum;
    }
};

```

• Maximum Sum Circular Subarray

```

int maxsum=nums[0];
int minsum=nums[0];
int currmin=0;
int currmax=0;
int n=nums.size();
int total=0;
for(int i=0;i<n;i++){
    total+=nums[i];
    currmax=max(nums[i],currmax+nums[i]);
    maxsum=max(maxsum,currmax);

    currmin=min(nums[i],currmin+nums[i]);
    minsum=min(minsum,currmin);
}
if(maxsum<0){
    return maxsum;
}
return max(maxsum,total-minsum);

```

• Maximum Product Subarray

```

int maxProduct(vector<int>& nums) {
    int n=nums.size();

```

- `int pre=1;`
- `int suf=1;`
- `int ans=INT_MIN;`
- `for(int i=0;i<n;i++){`
- `if(suf==0) suf=1;`
- `if(pre==0) pre=1;`
- `pre*=nums[i];`
- `suf*=nums[n-i-1];`
- `ans=max(ans,max(pre,suf));`
- `}`
- `return ans;`
- `}`

Maximum Subarray Sum with One Deletion

- `class Solution {`
- `public:`
- `int maximumSum(vector<int>& nums) {`
- `int n=nums.size();`
- `vector<int>pre(n);`
- `vector<int>suf(n);`
- `pre[0]=nums[0];`
- `for(int i=1;i<n;i++){`
- `pre[i]=max(nums[i],pre[i-1]+nums[i]);`
- `}`
- `suf[n-1]=nums[n-1];`
- `for(int i=n-2;i>=0;i--){`
- `suf[i]=max(nums[i],suf[i+1]+nums[i]);`
- `}`
- `//(left part ending at i-1) + (right part starting at i+1)`
- `int ans=*max_element(pre.begin(),pre.end());`
- `for(int i=1;i<n-1;i++){`
- `ans=max({ans,pre[i-1]+suf[i+1]});`
- `}`
- `return ans;`
- `}`
- `};`